1. CasinoShiz: casino bot mathematical model

1.1. 1. Notation

- `B_t` - player balance before the operation.
- `s` - player stake.
- `p` - player payout after the outcome.
- `m` - payout multiplier, so `p = s * m`.
- `RTP = E[p] / s` - return-to-player without external bonuses.
- `edge = 1 - RTP` - expected share of the stake retained by the system.
- `net = p - s` - player's net result.

Most games:

```
B_after = B_before - s + p
E[net] = E[p] - s = s * (RTP - 1)
```

Debit and credit operations are idempotent through operation ids.

1.2. 2. Base coin economy

1.2.1. 2.1. Stakes and payouts

For games with an upfront stake:

```
debit(s, reason = "<game>.bet")
roll outcome
credit(p, reason = "<game>.payout") if p > 0
```

The stake is limited by `MaxBet` in the specific game's settings.

1.2.2. 2.2. Transfers

Transfer sends `n` coins to the recipient and debits the sender:

```
fee_raw = n * FeePercent
fee_half = round_away_from_zero(fee_raw * 2) / 2
fee = max(MinFeeCoins, round_away_from_zero(fee_half))
total_debit = n + fee
```

Default: `FeePercent = 0.03`, `MinFeeCoins = 1`. The transfer fee is a coin sink.

1.2.3. 2.3. Daily bonus

```
bonus = min(MaxBonus, floor(balance * PercentOfBalance / 100))
```

Default: `PercentOfBalance = 0.35`, `MaxBonus = 8`, `MaxCatchUpDays = 14`. This is a coin faucet capped at 8 coins per local day.

1.2.4. 2.4. Gas tax and bank tax

```
GasDefault = 0.0285
GasModifier = sqrt(2)
GasFunction(x) = max(1, ((x + 1) ^ log10(x + 1) - 1) / 39.15)

if x < 10:
  gas(x) = round(GasFunction(x) * GasModifier)
else:
  gas(x) = round(x * GasDefault * GasModifier)
```

Current behavior: `gas(9) = 1`, but `gas(10) = 0` because of rounding in the second branch.

```
if bank < 70:       bank_tax = -2
else if bank < 120: bank_tax = 0
else:               bank_tax = round(max(4, min(gas(floor(bank)), 25)) / 2)
```

1.3. 3. Per-game model

1.3.1. 3.1. Telegram dice games

Telegram dice outcomes are treated as uniformly distributed.

Module	Outcomes	Win multipliers	RTP
`dicecube`	`1..6`	`4 -> x1, 5 -> x2, 6 -> x2`	`5/6 = 83.3333%`

darts	1..6	4 -> x1, 5 -> x2, 6 -> x2	5/6 = 83.3333%
bowling	1..6	4 -> x1, 5 -> x2, 6 -> x2	5/6 = 83.3333%
football	1..5	4 -> x2, 5 -> x2	4/5 = 80%
basketball	1..5	4 -> x2, 5 -> x2	4/5 = 80%

For `dicecube`, multipliers `Mult4`, `Mult5`, and `Mult6` are configurable:

```
RTP = (Mult4 + Mult5 + Mult6) / 6
```

With defaults `1,2,2`: `RTP = 83.3333%`, `edge = 16.6667%`.

1.3.2. 3.2. Slots (`/dice`, Telegram slot machine)

Current `DiceOptions.Cost = 9`. Gas is added on top:

```
loss = Cost + gas(Cost)
```

For the default `gas(9) = 1`, so `loss = 10`.

The Telegram slot value is decoded into 3 reels with 4 symbols:

```
roll_0 = ((value - 1) >> 0) & 3
roll_1 = ((value - 1) >> 2) & 3
roll_2 = ((value - 1) >> 4) & 3
```

index	symbol	stake price
0	bar	1
1	cherry	1
2	lemon	2
3	seven	3

Payout:

```
rolls_sum = sum(stake_price[roll_i])

three seven  -> 77
three lemon  -> 30
three cherry -> 23
three bar    -> 21
two seven    -> 10 + rolls_sum
two lemon    -> 6 + rolls_sum
any other pair -> 4 + rolls_sum
no pair      -> rolls_sum - 3
```

Across all 64 equally likely outcomes:

```
sum(payouts) = 610
E[p] = 610 / 64 = 9.53125
```

For default `loss = 10`:

```
RTP = 9.53125 / 10 = 95.3125%
edge = 4.6875%
E[net] = -0.46875 coins per spin
```

1.3.3. 3.3. Blackjack

Settlement rules:

```
if player_total > 21: payout = 0
else if player_blackjack and not dealer_blackjack: payout = bet + floor(3 * bet / 2)
else if player_blackjack and dealer_blackjack: payout = bet
else if dealer_total > 21: payout = 2 * bet
else if player_total > dealer_total: payout = 2 * bet
else if player_total < dealer_total: payout = 0
else: payout = bet
```

Natural blackjack pays `2.5x` the stake with integer `floor(1.5 * bet)` in the bonus part. There is no closed-form RTP in the code: it depends on the player's hit/stand/double strategy.

1.3.4. 3.4. Pick

The user defines `N` variants and chooses `k` backed variants.

```
P(win) = k / N
gross = s * N / k
base_payout = floor(gross * (1 - HouseEdge))
```

Default: `HouseEdge = 0.05`, `MinVariants = 2`, `MaxVariants = 10`, `MaxBet = 5000`.

Without streak bonus:

```
E[p] ~= s * (1 - HouseEdge)
RTP ~= 95%
edge ~= 5%
```

Streak bonus:

```
factor = min(max(0, streak_after - 1), StreakCap)
bonus = floor(s * factor * StreakBonusPerWin)
total_credit = base_payout + bonus
```

Default: `StreakBonusPerWin = 0.5`, `StreakCap = 4`, `ChainMaxDepth = 5`. On a long winning streak, the streak bonus can exceed the house edge.

1.3.5. 3.5. Pick lottery

```
pot = sum(stake_i)
if entrants < MinEntrantsToSettle:
  refund all stakes
else:
  winner ~ Uniform(entries)
  fee = floor(pot * HouseFeePercent)
  payout = pot - fee
```

Default: `MinEntrantsToSettle = 2`, `HouseFeePercent = 0.05`. With equal stakes, RTP is around 95%.

1.3.6. 3.6. Horse race

For horse `h`:

```
S_h = total stake on horse h
P = total pot

if S_h = 0:
  k_h = 1.0
else:
  k_h = floor(((P - S_h) / (1.1 * S_h)) * 1000) / 1000 + 1
```

If `h` wins:

```
payout_i = floor(stake_i * k_h)
total_payout_h ~= S_h + (P - S_h) / 1.1
house_hold_h ~= (P - S_h) / 11
```

Hold depends on stake distribution and winner. Full EV requires separately checking the distribution of `SpeedGenerator.GenPlaces(HorseCount)`.

1.4. 4. Meta progression

1.4.1. 4.1. XP per game

The formula is read from `meta_seasons.config.xp`. If JSON is missing or invalid, defaults are used:

```
play = 5
win = 25
loss = 2
stakeMultiplier = 0.01
minXpPerGame = 1
maxXpPerGame = 500
```

Calculation:

```
base_xp = is_win ? config.xp.win : config.xp.loss
stake_xp = floor(max(0, stake) * config.xp.stakeMultiplier)
xp_delta = clamp(config.xp.play + base_xp + stake_xp, minXpPerGame, maxXpPerGame)
```

With defaults:

```
win:  xp = clamp(30 + floor(0.01 * stake), 1, 500)
loss: xp = clamp(7  + floor(0.01 * stake), 1, 500)
```

1.4.2. 4.2. Rating

Rating is read from `meta_seasons.config.rating`:

```
enabled = true
start = 1000
winDelta = 16
lossDelta = -12
```

Calculation:

```
rating_delta = enabled ? (is_win ? winDelta : lossDelta) : 0
rating_after = max(0, rating_before + rating_delta)
```

Initial rating on the first record:

```
rating_initial = max(0, start + rating_delta)
```

1.4.3. 4.3. Level curve

Level curve is read from `meta_seasons.config.levels`:

```
xpPerLevelSquaredBase = 100
```

```
level(xp) = max(1, floor(sqrt(xp / xpPerLevelSquaredBase)) + 1)
xp_floor(level) = xpPerLevelSquaredBase * (level - 1)^2
```

level	minimum XP
1	0
2	100
3	400
4	900
5	1600
10	8100

1.4.4. 4.4. Seasons

```
DefaultDurationDays = 14
DefaultPreparedSeasonCount = 30
```

Runtime:

1. expired active season is moved to `finished`;
2. planned season that covers `now` is moved to `active`;
3. if there is no planned season, a new 14-day active season is created;
4. the planned seasons queue is filled up to 30 future seasons.

`SeasonPlanFactory` themes: `balanced`, `quest_rush`, `high_roller`, `clan_wars`, `tournament_arc`, `jackpot_hunt`, `streak_sprint`, `risk_watch`.

XP, rating start, rating delta, and level curve are applied from the season JSON config.

Season rewards are also read from the season JSON config:

```
rewards.playerTop = [5000, 2500, 1000]
rewards.clanTop   = [10000, 5000, 2500]
```

Actual payout by place:

```
reward(place) =
  rewards[place - 1], if place is within the array
  0, otherwise
```

Automatic rollover closes the expired active season, activates a matching planned season, and refills the future season queue. Rewards for finished seasons are processed idempotently through operation ids, so a repeated job must not duplicate payouts.

1.5. 5. Quest rotation

The quest catalog is stored in `games/Games.Meta/Infrastructure/Catalog/quest-pool.json`.

The active rotation is selected deterministically:

```
seed = season_id : chat_id : user_id : period_key : slot_id : index
```

Periods:

```
daily  -> yyyy-MM-dd
weekly -> ISO yyyy-Www
```

Rarity weights:

rarity	weight
common	100
uncommon	60
rare	25
epic	10
legendary	4

`meta_seasons.config.quests.focus` narrows the candidate pool when matching quests exist:

```
all-round / balanced / normal -> no narrowing
daily / weekly                -> quests for the selected period
volume                        -> volume/high_stake
payout                        -> payout/profit/multiplier
streaks                       -> streak tags/clusters
clans                         -> clan tags
tournaments                   -> tournament tags
controlled                    -> low_stake/play/loss
```

`meta_seasons.config.quests.rarityBias` changes effective weight:

```
normal   -> no change
uncommon -> uncommon * 2
rare     -> rare/epic/legendary * 3
epic     -> epic/legendary * 4
common   -> common * 2, others / 2
```

The progress gate remains the main constraint: rare or heavy quests will not be assigned to a new player if they specify `MinLevel`, `MinGamesPlayed`, or `MinTotalStaked`.

Progress gate:

```
quest unlocked iff
  player.level >= MinLevel
  and player.games_played >= MinGamesPlayed
  and player.total_staked >= MinTotalStaked
```

Quest progress delta:

```
volume -> stake
payout -> payout
profit -> max(0, payout - stake)
other  -> 1
```

1.6. 6. Faucets, sinks, and drift

Main sinks: negative EV dice games, transfer fee, pick/pick lottery rake, horse pool hold, gas tax on slots.

Main faucets: daily bonus, quest rewards, season rewards, clan rewards, redeem drops, Pick streak bonus.

For period `T`:

```
DeltaSupply(T) =
  sum_game_payouts(T)
  + sum_daily_bonus(T)
  + sum_quest_coin_rewards(T)
  + sum_season_rewards(T)
  + sum_redeem_rewards(T)
  - sum_stakes_debited(T)
  - sum_transfer_fees(T)
  - sum_lottery_fees(T)
```

Stable economy condition:

```
daily_bonus + quest_coins + season_rewards + redeem_value
  <= expected_house_edge + transfer_fees + lottery_fees
```

The admin page `/admin/meta-economy` calculates a rough simulation for selected `days`, `players`, average stake/RTP, and seasonal reward pool, then shows actual ledger drift from `economics_ledger` next to it. This is a quick sanity check, not Monte Carlo.

1.7. 7. Quick balance summary

Module	RTP / effect	Edge / sink
DiceCube	83.3333%	16.6667%
Darts	83.3333%	16.6667%
Bowling	83.3333%	16.6667%
Football	80.0000%	20.0000%
Basketball	80.0000%	20.0000%
Slots default	95.3125%	4.6875%
Pick base	around 95%	around 5%
Pick lottery	around 95%	around 5%
Horse	depends on pool	`(P - S_winner) / 11`
Transfer	no RTP	3% fee, minimum 1
Daily bonus	faucet	up to 8 coins/day